

Introduction to **Information Retrieval**

Lecture 16: Web search basics

Duplicate detection

Duplicate documents

- The web is full of duplicated content
- Strict duplicate detection = exact match
 - Not as common
- But many, many cases of near duplicates
 - E.g., Last modified date the only difference between two copies of a page

Duplicate/Near-Duplicate Detection

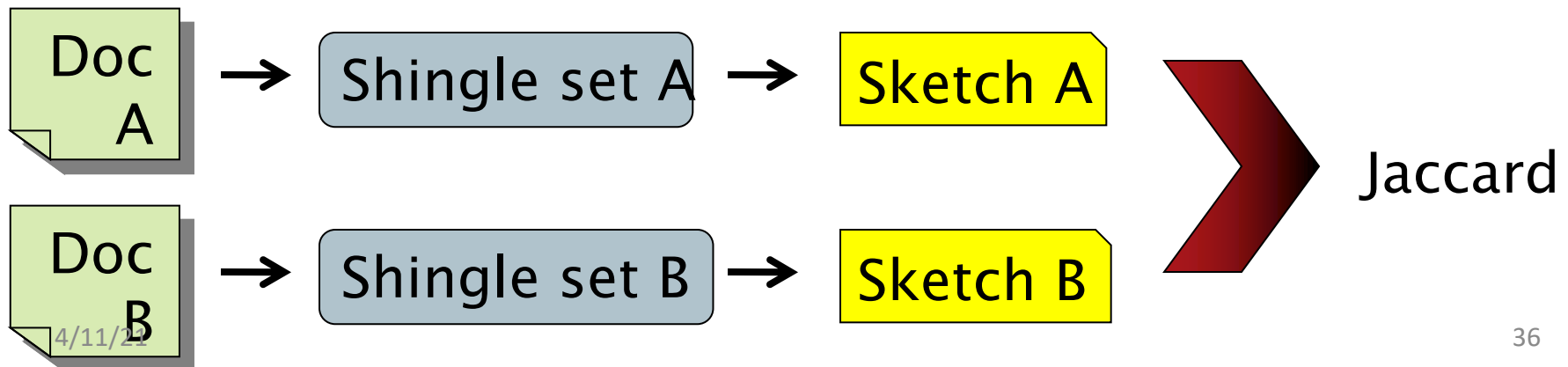
- *Duplication*: Exact match can be detected with fingerprints
- *Near-Duplication*: Approximate match
 - Overview
 - Compute syntactic similarity with an edit-distance measure
 - Use similarity threshold to detect near-duplicates
 - E.g., Similarity > 80% => Documents are “near duplicates”
 - Not transitive though sometimes used transitively

Computing Similarity

- Features:
 - Segments of a document (natural or artificial breakpoints)
 - Shingles (Word N-Grams)
 - ***a rose is a rose is a rose*** →
 - a_rose_is_a
 - rose_is_a_rose
 - is_a_rose_is
 - a_rose_is_a
- Similarity Measure between two docs (= sets of shingles)
 - Set intersection
 - Specifically (Size_of_Intersection / Size_of_Union)

Shingles + Set Intersection

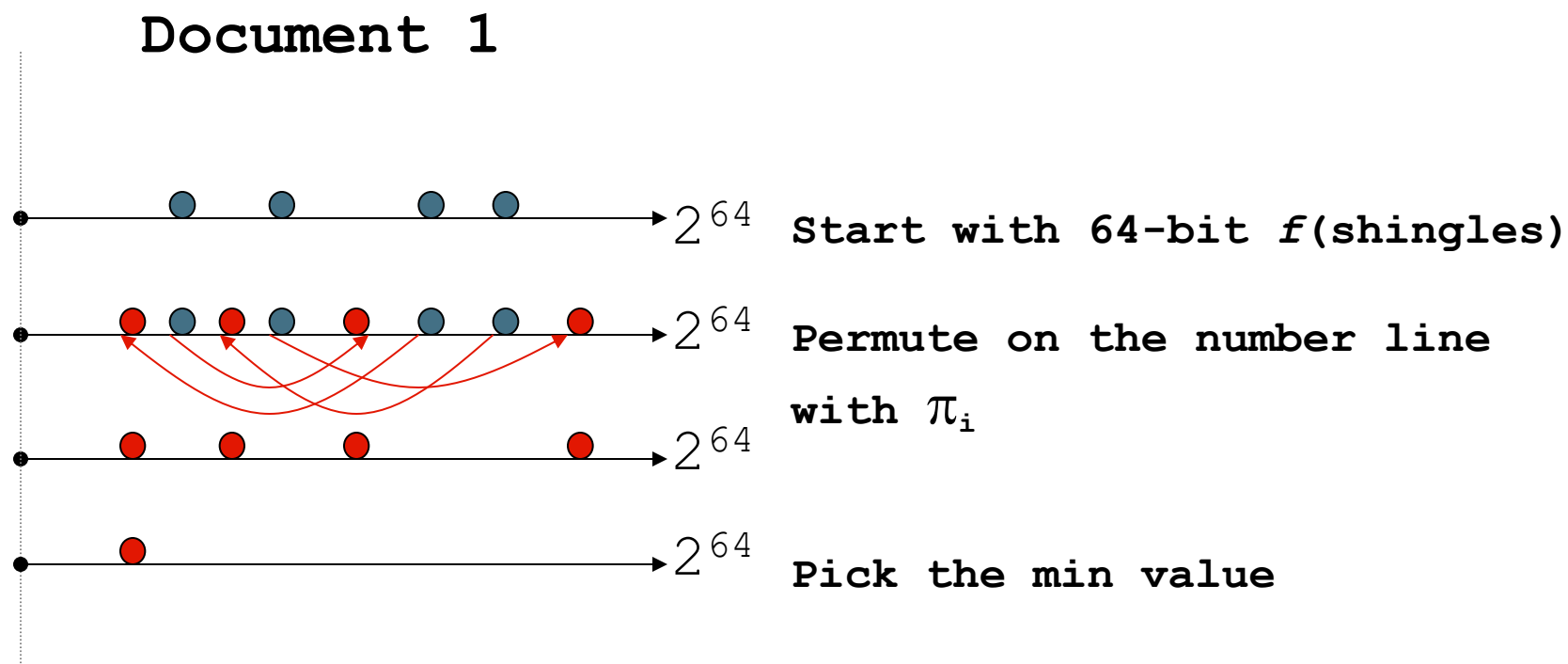
- Computing exact set intersection of shingles between all pairs of documents is expensive/intractable
 - Approximate using a cleverly chosen subset of shingles from each (a *sketch*)
- Estimate $(\text{size_of_intersection} / \text{size_of_union})$ based on a short sketch



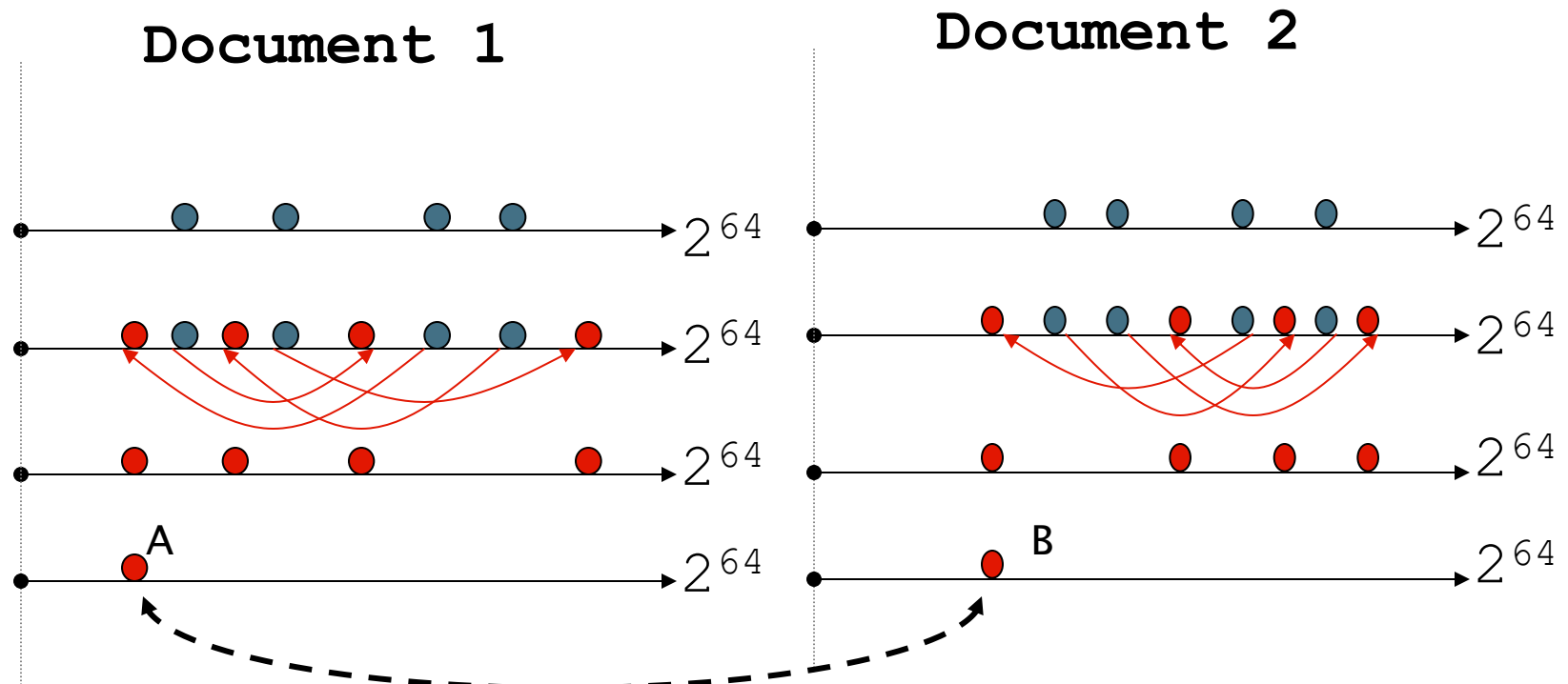
Sketch of a document

- Create a “sketch vector” (of size ~ 200) for each document
 - Documents that share $\geq t$ (say 80%) corresponding vector elements are **near duplicates**
 - For doc D , $\text{sketch}_D[i]$ is as follows:
 - Let f map all shingles in the universe to $[0, 2^m-1]$ (e.g., f = fingerprinting)
 - Let π_i be a *random permutation* on $[0, 2^m-1]$
 - Pick $\text{MIN } \{\pi_i(f(s))\}$ over all shingles s in D

Computing Sketch[i] for Doc1



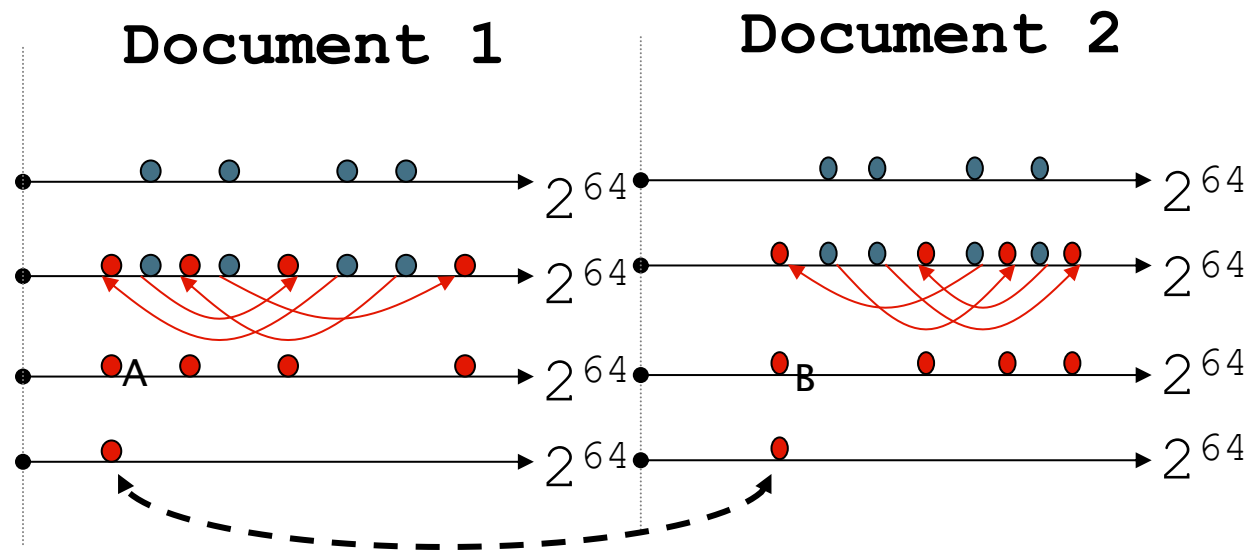
Test if $\text{Doc1.Sketch}[i] = \text{Doc2.Sketch}[i]$



Are these equal?

Test for **200** random permutations: $\pi_1, \pi_2, \dots, \pi_{200}$

However...



A = B iff the shingle with the MIN value in the union of Doc1 and Doc2 is common to both (i.e., lies in the intersection)

Claim: This happens with probability

$$\text{Size_of_intersection} / \text{Size_of_union}$$

Why?

Set Similarity of sets C_i, C_j

$$\text{Jaccard}(C_i, C_j) = \frac{|C_i \cap C_j|}{|C_i \cup C_j|}$$

- View sets as columns of a matrix A ; one row for each element in the universe. $a_{ij} = 1$ indicates presence of item i in set j

- Example

C_1	C_2
0	1
1	0
1	1
0	0
1	1
0	1

$$\text{Jaccard}(C_1, C_2) = 2/5 = 0.4$$

Key Observation

- For columns C_i, C_j , four types of rows

	C_i	C_j
type A	1	1
type B	1	0
type C	0	1
type D	0	0

- Overload notation: A = # of rows of type A
- Claim

$$\text{Jaccard}(C_i, C_j) = \frac{A}{A + B + C}$$

“Min” Hashing

- Randomly **permute** rows
- **Hash** $h(C_i)$ = index of first row with 1 in column C_i
- **Surprising Property**
$$\Pr [h(C_i) = h(C_j)] = \text{Jaccard}(C_i, C_j)$$
- **Why?**
 - Both are $A/(A+B+C)$
 - Look down columns C_i, C_j until first **non-Type-D** row
 - $h(C_i) = h(C_j) \leftrightarrow$ type A row

Min-Hash sketches

- Pick P random row permutations
- MinHash sketch

$\text{sketch}(C)$ = list of k indexes of first rows with 1 in column C

↑
this is a random variable

- Similarity of signatures
 - Let $\text{sim}[\text{sketch}(C_i), \text{sketch}(C_j)]$ = fraction of permutations where MinHash values agree
 - Observe $\mathbb{E}[\text{sim}(\text{sig}(C_i), \text{sig}(C_j))] = \text{Jaccard}(C_i, C_j)$

Practical Implementation

- Random permutation is hard to obtain; simulate them using universal hashing instead
 - $h: \{0, 1, 2, \dots, U\} \rightarrow \{0, 1, 2, \dots, M\}$
 - $h(x) = ((a * x + b) \bmod P) \bmod M$
 - where
 - $P \gg U$ and is a prime number
 - a, b are two randomly chosen integers modulo P and $a \neq 0$
 - $\text{sketch}(C) = \{ \text{argmin}_{e \in C} \{ h_i(e) \} \mid 1 \leq i \leq k \}$

Example

	C_1	C_2	C_3
R_1	1	0	1
R_2	0	1	1
R_3	1	0	0
R_4	1	0	1
R_5	0	1	0

Signatures

	S_1	S_2	S_3
Perm 1 = (12345)	1	2	1
Perm 2 = (54321)	4	5	4
Perm 3 = (34512)	3	5	4

Similarities

	1-2	1-3	2-3
Col-Col	0.00	0.50	0.25
Sig-Sig	0.00	0.67	0.00

Example Using the Universal Hashing

$$h(x) = (7x+1 \bmod 31) \bmod 9$$

$$g(x) = (17x+8 \bmod 31) \bmod 9$$

	C_1	C_2	C_3
R_1	1	0	1
R_2	0	1	1
R_3	1	0	0
R_4	1	0	1
R_5	0	1	0

Note: this example results in different sketches from the previous slide

$$S_1 = \{R_1, R_3, R_4\}$$

$$h(e) = \{8, 4, 2\} \rightarrow \min_elem = R_4$$

$$g(e) = \{7, 1, 5\} \rightarrow \min_elem = R_3$$

$$\text{sketch}(S_1) = \{R_4, R_3\}$$

$$S_2 = \{R_2, R_5\}$$

$$h(e) = \{6, 5\} \rightarrow \min_elem = R_5$$

$$g(e) = \{2, 0\} \rightarrow \min_elem = R_5$$

$$\text{sketch}(S_2) = \{R_5, R_5\}$$



Therefore, estimated similarity between S_1 and S_2 is $0/3 = 0.0$

All signature pairs

- Now we have an extremely efficient method for estimating a Jaccard coefficient for a single pair of documents.
- But we still have to estimate N^2 coefficients where N is the number of web pages.
 - Still slow
- One solution: locality sensitive hashing (LSH)
- Another solution: Sorting (Henzinger 2006)

More resources

- IIR Chapter 19